

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ЛАБОРАТОРНАЯ РАБОТА №2
по дисциплине «Вычислительная математика»
Тема: Решение систем линейных алгебраических уравнений

Студентка гр. 4342

Киселева А.Д.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы

Целью лабораторной работы является изучение методов Гаусса и Якоби для решения систем линейных уравнений, написание программы, реализующей данные методы с заданной точностью, тестирование программы на квадратных матрицах различного размера (от 10 до 10 000), в том числе на плохо обусловленных матрицах (например, матрицах Гильберта), сравнение полученных решений с решениями, полученными с помощью встроенных функций языка программирования (библиотеки Lapacke).

Теоретический минимум.

Метод Гаусса — это классический алгоритм последовательного исключения неизвестных для решения систем линейных уравнений. Он приводит матрицу системы к треугольному виду с помощью элементарных преобразований, после чего переменные находятся обратной подстановкой. Метод универсален и подходит для систем с любым числом решений.

Существуют модификации алгоритма для повышения точности:

- Метод частичного выбора — на каждом шаге выбирается максимальный элемент из столбца;
- Метод полного выбора — на каждом шаге выбирается максимальный элемент из всех элементов по индексу больших текущего.

Для оценки погрешности полученного приближенного решения $\bar{x}$ по сравнению с точным (эталонным) решением x^* используется Евклидова норма вектора ошибки:

$$e = \|\bar{x} - x^*\|_2 = \sqrt{\sum_{i=1}^n (\bar{x}_i - x_i^*)^2}$$

Метод Якоби — это итерационный метод решения систем линейных алгебраических уравнений. Из каждого уравнения системы выражается

соответствующая неизвестная переменная через остальные. Полученные формулы используются для построения итерационного процесса, где каждое новое приближение вычисляется на основе предыдущего. Для системы n уравнений с n неизвестными на каждом шаге итерации новое значение i -й переменной вычисляется по формуле:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right),$$

где k — номер итерации, a_{ii} — диагональный элемент матрицы, b_i — компонента вектора правой части.

Метод Якоби сходится, если выполнено условие диагонального преобладания:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad i = 1..n,$$

то есть модуль каждого диагонального элемента должен быть строго больше суммы модулей остальных элементов в соответствующей строке матрицы.

Выполнение работы.

Метод Гаусса.

Создана отдельная функция для генерации квадратной матрицы размера $N \times N$, а также вектора правой части. Заполнение матрицы выполняется с помощью случайного равномерного распределения на интервале $[0.0; 10.0]$ через генератор псевдослучайных чисел `std::mt19937`.

В качестве модификации метода Гаусса был выбран метод частичного выбора, трудоёмкость которого составляет $O\left(\frac{2n^3}{3}\right)$ операций, что является оптимальным балансом между скоростью работы и вычислительной устойчивостью. На каждом шаге прямого хода метода частичного выбора выбирается максимальный по модулю элемент в текущем столбце, после чего происходит перестановка строки текущего ведущего элемента с найденной строкой.

Проверка матрицы на вырожденность встроена непосредственно в процесс прямого хода метода Гаусса. Если при поиске ведущего элемента максимальное значение в столбце оказывается близким к нулю (меньше заданного порога $\varepsilon = 10^{-12}$), это свидетельствует о линейной зависимости строк ($\det A \approx 0$). В таком случае программа прерывает вычисления, выводя сообщение об ошибке.

Метод Якоби.

Как и для метода Гаусса, создана отдельная функция для генерации матрицы размера $N \times N$ и вектора правой части, заполненная случайными значениями в диапазоне $[0.0; 10.0]$. Для обеспечения диагонального преобладания диагональные элементы генерируются отдельно на интервале $[10N, 11N]$, что гарантирует выполнение условия сходимости метода Якоби.

Перед началом итерационного процесса вызывается функция `checkDiagonalDominance()`, которая проверяет достаточное условие сходимости метода. Если условие не выполняется хотя бы для одной строки, метод Якоби считается неприменимым, и программа выводит соответствующее сообщение.

На каждой итерации для каждой переменной вычисляется новое значение по соответствующему уравнению, используя только старые значения остальных переменных с предыдущего шага. Все переменные обновляются одновременно, после чего процесс повторяется до достижения нужной точности ($\varepsilon = 10^{-2}$).

Тестирование.

Метод Гаусса.

Для проверки решения используется библиотека Lараске, в которую передается сгенерированная матрица. Оценка корректности вычисляется как отношение решения по Гауссу к решению с помощью Lараске по 1 и 2-норме.

Для проверки устойчивости метода частичного выбора программа протестирована на матрицах Гильберта, которые являются плохо обусловленными (число обусловленности растёт экспоненциально с ростом N).

Результаты тестирования на матрицах различного размера приведены в таблицах 1 и 2.

Таблица 1 - Тестирование на случайных матрицах

Размер матрицы	Оценка по 1-норме	Оценка по 2-норме
10	3.38278e-15	3.39009e-15
100	8.31956e-15	8.31956e-15
1000	1.18331e-13	1.16853e-13
10000	1.8383e-11	1.83436e-11

Таблица 2 - Тестирование на матрицах Гильберта

Размер матрицы	Оценка по 1-норме	Оценка по 2-норме
8	2.65968e-09	2.67204e-09
9	7.35547e-08	7.34549e-08
10	1.54739e-06	1.55105e-06
11	Система не имеет решений или их бесконечное количество	

Метод Якоби.

Проверка решения также осуществляется с помощью библиотеки Лараске посредством сравнения результатов по 1 и 2-норме. Результаты тестирования приведены в таблице 3.

Таблица 3 - Тестирование на матрицах с диагональным преобладанием

Размер матрицы	Кол-во итераций	Оценка по 1-норме	Оценка по 2-норме
10	5	0.0267195	0.0267195
100	4	0.0466093	0.0395687
1000	2	0.209746	0.172312
10000	1	0.442623	0.362236

Вывод

В результате выполнения лабораторной работы были изучены методы Гаусса и Якоби, написаны программы, находящие решения СЛАУ с заданной точностью. Проведено тестирование на квадратных матрицах различных размеров (до 10 000), проведено сравнение полученных результатов с результатами, полученными при помощи функций библиотеки Lapacke, проанализировано поведение на плохо обусловленных матрицах.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД

Файл gauss.cpp

```
#include <iostream>
#include <random>
#include <vector>
#include <cmath>
#include <lapacke.h>

int N;
const int n = 10000;
double matrix[n][n+1];
double eps = 1e-12;

void generateMatrix(){
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<double> dist(0.0, 10.0);
//Равномерное распределение

    for (int i = 0; i < N; ++i){
        for (int j = 0; j < (N+1); ++j){
            matrix[i][j] = dist(gen);
        }
    }
}

bool forwardPhase(){
    int currentRow = 0;    //Номер строки, где должен лежать
    следующий опорный элемент

    for (int col = 0; col < N; ++col){
        int pivotRow = currentRow;
        double maxVal = fabs(matrix[currentRow][col]);

        for (int row = currentRow + 1; row < N; ++ row){    //Поиск
            максимального элемента в столбце
            if (fabs(matrix[row][col]) > maxVal){
                maxVal = fabs(matrix[row][col]);
                pivotRow = row;
            }
        }

        if (maxVal < eps) return false;

        if (pivotRow != currentRow){    //Перестановка строк, если
            максимальный элемент не был в опорной строке
            for (int i = 0; i < N + 1; ++i){
                double temp = matrix[currentRow][i];
                matrix[currentRow][i] = matrix[pivotRow][i];
                matrix[pivotRow][i] = temp;
            }
        }
    }
}
```

```

    }

    for (int row = currentRow + 1; row < N; ++row){
        double mult = matrix[row][col]/matrix[currentRow][col];
        for (int j = col; j < N + 1; ++j){
            matrix[row][j] -= mult * matrix[currentRow][j];
        }
    }

    currentRow++;
}

return true;
}

std::pair<double, double> calcNormas(std::vector<double>& ansGauss,
std::vector<double>& ansLapacke){
    double oneNorma = 0.0;
    double twoNorma = 0.0;
    double oneNormaLapacke = 0.0;
    double twoNormaLapacke = 0.0;

    for (int i = 0; i < N; ++i){
        oneNorma += fabs(ansLapacke[i] - ansGauss[i]);
        twoNorma += (ansLapacke[i] - ansGauss[i]) * (ansLapacke[i]
- ansGauss[i]);
        oneNormaLapacke += fabs(ansLapacke[i]);
        twoNormaLapacke += ansLapacke[i] * ansLapacke[i];
    }

    return {oneNorma / oneNormaLapacke, sqrt(twoNorma) /
sqrt(twoNormaLapacke)};
}

std::vector<double> backwardPhase(){
    std::vector<double> ans(N);

    for (int i = N-1; i >= 0; --i){
        double sum = 0.0;
        for (int j = i+1; j < N; ++j){
            sum += matrix[i][j] * ans[j];    //Вычисление суммы
произведений коэффициентов и уже найденных X
        }
        ans[i] = (matrix[i][N] - sum) / matrix[i][i];
    }

    return ans;
}

bool gauss(std::vector<double>& ans){
    if (!forwardPhase()){
        std::cout << "Система не имеет решений или их бесконечное
количество.\n";
        return false;
    }
}

```



```

        ans = backwardPhase();
        return true;
    }

    void lapack(std::vector<double>& A, std::vector<double>& b){
        int ipiv[N];
        int info = LAPACKE_dgesv(
            LAPACK_ROW_MAJOR, // порядок хранения (по строкам)
            N,                 // размер матрицы
            1,                 // одна правая часть
            A.data(),          // матрица A
            N,                 // ведущая размерность
            ipiv,              // pivot-индексы
            b.data(),          // правая часть (на выходе здесь
будет решение)
            1                  // ведущая размерность b
        );
    }

    void compareGaussLapacke(std::vector<double>& matrixForLapacke,
std::vector<double>& rightPart){
        std::vector<double> ansGauss(N);
        if (gauss(ansGauss)){
            lapack(matrixForLapacke, rightPart);

            std::pair<double, double> normas = calcNormas(ansGauss,
rightPart);

            std::cout << "Относительная погрешность метода Гаусса по 1-
норме = " << normas.first << '\n';
            std::cout << "Относительная погрешность метода Гаусса по 2-
норме = " << normas.second << '\n';
            std::cout << "За точное взято решение с помощью Lapacke\n\
n";
        }
    }

    int main(){
        std::cout << "КОМПИЛЯЦИЯ: g++ gauss.cpp -o gauss -llapacke -
llapack -lblas\n";
        std::cout << "Введите размер матрицы <= 10 000\n";
        std::cin >> N;

        generateMatrix();

        std::vector<double> matrixForLapacke(N*N);
        std::vector<double> rightPart(N);

        for(int i = 0; i < N; ++i){
            for(int j = 0; j < N+1; ++j){
                if (j == N) rightPart[i] = matrix[i][j];
                else matrixForLapacke[i*N+j] = matrix[i][j];
            }
        }
    }

```

```

std::cout << "Случайная матрица размера " << N << "\n\n";

compareGaussLapacke(matrixForLapacke, rightPart);

std::random_device rd;
std::mt19937 gen(rd());
std::uniform_real_distribution<double> dist(0.0, 10.0);

for (int i = 0; i < N; ++i){
    for (int j = 0; j < N+1; ++j){
        if (j != N){
            matrix[i][j] = 1.0 / (i + j + 1);    //Матрица
Гильберта
            matrixForLapacke[i*N+j] = matrix[i][j];
        } else {
            matrix[i][j] = dist(gen);
            rightPart[i] = matrix[i][j];
        }
    }
}

std::cout << "Матрица Гильберта размера " << N << "\n\n";

compareGaussLapacke(matrixForLapacke, rightPart);

return 0;
}

```

Файл jackobi.cpp

```

#include <iostream>
#include <random>
#include <vector>
#include <cmath>
#include <lapacke.h>

int N;
const int n = 15000;
double matrix[n][n+1];
double eps = 1e-2;

void generateMatrix(){
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<double> dist(0.0, 10.0);
//Равномерное распределение
    std::uniform_real_distribution<double> distDiag(10.0*N,
11.0*N);

    for (int i = 0; i < N; ++i){
        for (int j = 0; j < (N+1); ++j){
            if (i == j) matrix[i][j] = distDiag(gen);
            else matrix[i][j] = dist(gen);
        }
    }
}

```

```

    }

    double normaForJakobi(std::vector<double>& ansPrev,
std::vector<double>& ansCurr){
        double norma = 0.0;
        for (int i = 0; i < N; ++i){
            norma += (ansCurr[i] - ansPrev[i]) * (ansCurr[i] -
ansPrev[i]);
        }
        return sqrt(norma);
    }

    bool checkDiagonalDominance(){
        for (int i = 0; i < N; ++i){
            double sum = 0.0;

            for (int j = 0; j < N; ++j){
                if (j != i)
                    sum += fabs(matrix[i][j]);
            }

            if (fabs(matrix[i][i]) <= sum)
                return false;
        }

        return true;
    }

    bool jakobi(std::vector<double>& ans, int& cntIter){
        if (!checkDiagonalDominance()){
            std::cout << "Метод Якоби не применим для решения данной
системы.\n";
            return false;
        } else {
            std::vector<double> xOld(N);

            double residual = 10.0;
            cntIter = 0;

            while (residual > eps){
                std::vector<double> xCurr(N);

                for (int i = 0; i < N; ++i){
                    double b = matrix[i][N];
                    double sum = 0.0;

                    for (int j = 0; j < N; ++j){
                        if (i != j)
                            sum += matrix[i][j] * xOld[j];
                    }

                    xCurr[i] = (b - sum) / matrix[i][i];
                }

                residual = normaForJakobi(xOld, xCurr);
            }
        }
    }

```

```

        copy(xCurr.begin(), xCurr.end(), xOld.begin());
        cntIter++;
    }

    ans = xOld;
    return true;
}

}

void lapack(std::vector<double>& A, std::vector<double>& b){
    int ipiv[N];
    int info = LAPACKE_dgesv(
        LAPACK_ROW_MAJOR, // порядок хранения (по строкам)
        N,                 // размер матрицы
        1,                 // одна правая часть
        A.data(),          // матрица A
        N,                 // ведущая размерность
        ipiv,              // pivot-индексы
        b.data(),          // правая часть (на выходе здесь
будет решение)
        1                  // ведущая размерность b
    );
}

std::pair<double, double> calcNormas(std::vector<double>&
ansJakobi, std::vector<double>& ansLapacke){
    double oneNorma = 0.0;
    double twoNorma = 0.0;
    double oneNormaLapacke = 0.0;
    double twoNormaLapacke = 0.0;

    for (int i = 0; i < N; ++i){
        oneNorma += fabs(ansLapacke[i] - ansJakobi[i]);
        twoNorma += (ansLapacke[i] - ansJakobi[i]) * (ansLapacke[i]
- ansJakobi[i]);
        oneNormaLapacke += fabs(ansLapacke[i]);
        twoNormaLapacke += ansLapacke[i] * ansLapacke[i];
    }

    return {oneNorma / oneNormaLapacke, sqrt(twoNorma) /
sqrt(twoNormaLapacke)};
}

void compareJakobiLapacke(std::vector<double>& matrixForLapacke,
std::vector<double>& rightPart){
    std::vector<double> ansJakobi;
    int cntIterJacobi = 0;
    if (jakobi(ansJakobi, cntIterJacobi)){
        lapack(matrixForLapacke, rightPart);

        std::pair<double, double> normas = calcNormas(ansJakobi,
rightPart);

        std::cout << "Количество итераций для нахождения решения =
" << cntIterJacobi << '\n';
        std::cout << "Относительная погрешность метода Якоби по 1-
норме = " << normas.first << '\n';
    }
}

```

```

        std::cout << "Относительная погрешность метода Якоби по 2-
норме = " << normas.second << '\n';
        std::cout << "За точное взято решение с помощью Lapacke\n\
n";
    }

}

int main(){
    std::cout << "Введите размер матрицы <= 10 000\n";
    std::cin >> N;

    generateMatrix();

    std::vector<double> matrixForLapacke(N*N);
    std::vector<double> rightPart(N);

    for(int i = 0; i < N; ++i){
        for(int j = 0; j < N+1; ++j){
            if (j == N) rightPart[i] = matrix[i][j];
            else matrixForLapacke[i*N+j] = matrix[i][j];
        }
    }

    std::cout << "Случайная матрица размера " << N << "\n\n";

    compareJakobiLapacke(matrixForLapacke, rightPart);

    return 0;
}

```